DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2026-27
ODD SEMESTER

INFORMATION ASSURANCE AND SECURITY

(24CS3105)

ALM – PROTOTYPE REVIEW

submitted by
Luhitha Boppana 2420030469

Cluster: 4 Section: 11

Course Instructor
Dr. Jagan Mohan Reddy Danda
Associate Professor

KONERU LAKSHMAIAH EDUCATION FOUNDATION, BOWRAMPET

Hyderabad-500043, Telangana, India

2026-27

Abstract

This report presents the implementation and study of fundamental cryptographic algorithms as part of the Information Assurance and Security course. The prototype covers classical encryption techniques such as Caesar Cipher, Playfair Cipher, Hill Cipher and Vigenère Cipher, along with modern cryptographic techniques including Simple Data Encryption Standard (SDES), Advanced Encryption Standard (AES), Stream Cipher (RC4), Diffie–Hellman key exchange and RSA. For each algorithm, the report explains the underlying concept, mathematical notation, working procedure, algorithm or pseudocode, implementation details and experimental results. Screenshots of the implemented programs and their outputs are included to demonstrate the working of the prototypes. The report also provides the GitHub repository containing the source code and implementation files.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

Information Assurance and Security focuses on protecting information and communication systems from unauthorized access, modification, disclosure and disruption. Cryptography is one of the fundamental techniques used to achieve confidentiality, integrity, authentication and secure communication.

This prototype review demonstrates the implementation of multiple cryptographic algorithms. The algorithms selected include classical substitution and transposition techniques, symmetric-key algorithms, stream ciphers and public-key cryptographic techniques.

The major objectives of this prototype are:

- To understand the fundamental concepts of cryptography.
- To implement classical encryption and decryption algorithms.
- To understand symmetric-key cryptography.
- To understand public-key cryptography.
- To implement key exchange using Diffie–Hellman.
- To implement RSA encryption and decryption.
- To observe plaintext, ciphertext, keys and decrypted output.
- To document the implementation and experimental results.

1.1 Objectives

The objectives of the prototype are:

1. Study the working principles of different cryptographic algorithms.

2. Implement encryption and decryption procedures.
3. Represent cryptographic operations mathematically.
4. Test the algorithms using suitable input values.
5. Verify that encrypted data can be correctly decrypted.
6. Document implementation results with screenshots.

1.2 Cryptography

Cryptography is the science of protecting information by transforming readable information, called plaintext, into an unreadable form called ciphertext. The transformation is performed using an encryption algorithm and a key.

The reverse transformation of ciphertext into plaintext is called decryption.

The basic cryptographic model is:

$$C = E_K(P)$$

where:

- P = Plaintext
- C = Ciphertext
- E = Encryption function
- D = Decryption function
- K = Cryptographic key

Decryption is represented as:

$$P = D_K(C)$$

Chapter 2

Classical Cryptographic Algorithms

2.1 Caesar Cipher

2.1.1 Definition

The Caesar Cipher is a substitution cipher in which each letter of the plaintext is shifted by a fixed number of positions in the alphabet. It is one of the simplest classical encryption techniques.

2.1.2 Mathematical Notation

The encryption operation is:

$$C = (P + K) \bmod 26$$

The decryption operation is:

$$P = (C - K) \bmod 26$$

where P represents the plaintext character, C represents the ciphertext character and K represents the shift value.

2.1.3 Algorithm

Algorithm 1 Caesar Cipher Encryption

Require: Plaintext P, shift value K
Ensure: Ciphertext C

```
for each character p in P do
    if p is an alphabet then
        Convert p into numerical position
        c ← (p + K) mod 26
        Convert c back into a character
    else
        Keep character unchanged
    end if
end for
return C
```

2.1.4 Implementation

The Caesar Cipher was implemented to accept plaintext and a key value. The plaintext is encrypted using the selected shift value and the resulting ciphertext is displayed. The ciphertext is then decrypted using the same key.

2.1.5 Result

The encryption and decryption operations were successfully performed. The decrypted text matched the original plaintext.

Table 2.1: Caesar Cipher Result

Parameter	**Value**
Plaintext	HELLO
Key	3
Ciphertext	KHOOR
Decrypted Text	HELLO

2.1.6 Screenshot

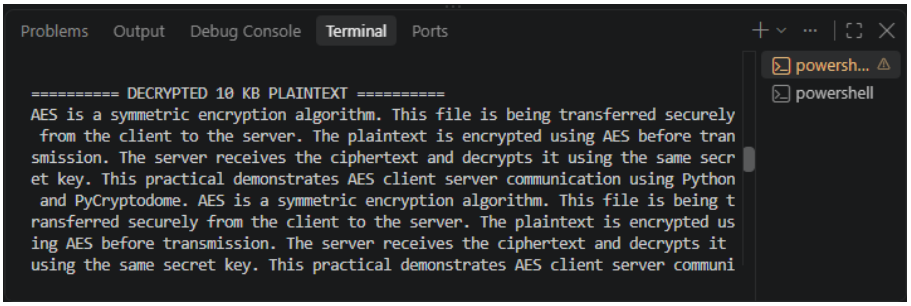

Figure 2.1: Caesar Cipher Implementation and Output

2.2 Playfair Cipher

2.2.1 Definition

The Playfair Cipher is a digraph substitution cipher that encrypts pairs of letters instead of individual characters. A 5×5 matrix is generated using a keyword.

2.2.2 Mathematical Notation

The plaintext is divided into pairs:

$$P = (P_1, P_2)$$

Each pair is encrypted according to its position in the Playfair matrix.

The major rules are:

- Same row: replace each letter with the letter to its right.
- Same column: replace each letter with the letter below it.
- Rectangle: replace each letter with the letter in the same row and the column of the other letter.

2.2.3 Algorithm

Algorithm 2 Playfair Cipher

Require: Plaintext P, keyword K
Ensure: Ciphertext C

```
Generate 5 × 5 key matrix using K
Prepare plaintext into digraphs
for each digraph (a, b) do
    if a and b are in the same row then
        Replace each with the character to its right
    else if a and b are in the same column then
        Replace each with the character below it
    else
        Apply rectangle rule
    end if
end for
return ciphertext
```

2.2.4 Implementation

The implementation generates the Playfair matrix using a keyword and processes the plaintext as character pairs.

2.2.5 Result

The plaintext was successfully converted into ciphertext using the Playfair matrix and was subsequently decrypted.

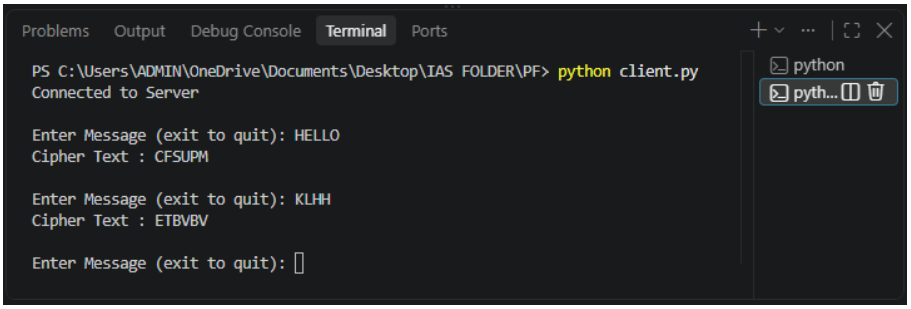

Figure 2.2: Playfair Cipher Implementation and Output

2.3 Hill Cipher

2.3.1 Definition

The Hill Cipher is a polygraphic substitution cipher based on matrix multiplication. It uses a key matrix to transform plaintext vectors into ciphertext vectors.

2.3.2 Mathematical Notation

Encryption is represented as:

$$C = KP \pmod{26}$$

where K is the key matrix, P is the plaintext vector and C is the ciphertext vector.

Decryption is:

$$P = K^{-1}C \pmod{26}$$

where K^{-1} is the modular inverse of the key matrix.

2.3.3 Algorithm

Algorithm 3 Hill Cipher

Require: Plaintext vector P, key matrix K
Ensure: Ciphertext C
1: Convert plaintext characters to numerical values
2: Divide plaintext into blocks
3: **for** each plaintext block P_i **do**
4: $C_i \leftarrow KP_i \bmod 26$
5: **end for**
6: **return** ciphertext

2.3.4 Result

The Hill Cipher successfully performed matrix-based encryption and decryption.

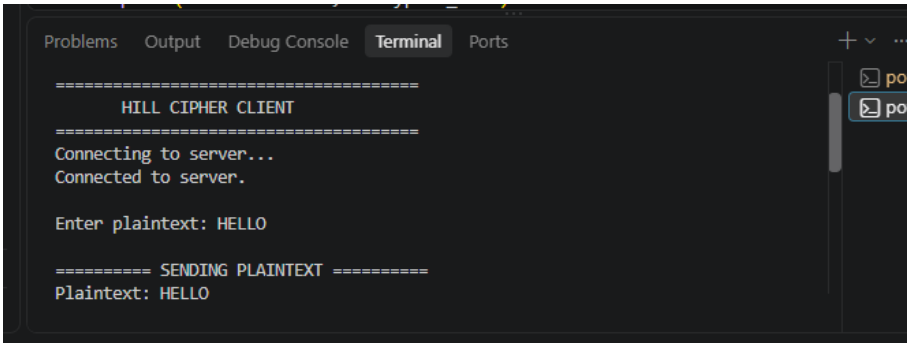

Figure 2.3: Hill Cipher Implementation and Output

2.4 Vigenère Cipher

2.4.1 Definition

The Vigenère Cipher is a polyalphabetic substitution cipher that uses a keyword to determine different shift values for different characters.

2.4.2 Mathematical Notation

Encryption:

$$C_i = (P_i + K_i) \bmod 26$$

Decryption:

$$P_i = (C_i - K_i) \bmod 26$$

2.4.3 Algorithm

Algorithm 4 Vigenère Cipher

Require: Plaintext P, keyword K
Ensure: Ciphertext C

1: Repeat the keyword until its length matches the plaintext
2: **for** $i = 1$ to length of plaintext **do**
3: Convert P_i and K_i to numerical values
4: $C_i \leftarrow (P_i + K_i) \bmod 26$
5: **end for**
6: **return** ciphertext

2.4.4 Result

The plaintext was successfully encrypted using the keyword and decrypted using the same key.

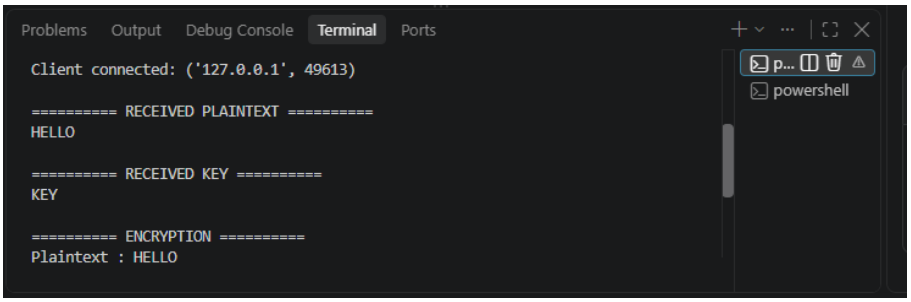

Figure 2.4: Vigenère Cipher Implementation and Output

Chapter 3

Symmetric-Key Cryptographic Algorithms

3.1 Simple Data Encryption Standard (SDES)

3.1.1 Definition

Simple Data Encryption Standard (SDES) is a simplified educational version of the Data Encryption Standard. It demonstrates the basic concepts of block cipher encryption including key generation, permutations, substitution and multiple rounds of processing.

3.1.2 Mathematical Notation

The general encryption model is:

$$C = E_K(P)$$

and decryption is:

$$P = D_K(C)$$

SDES uses permutations and substitution operations to transform the input plaintext.

3.1.3 Algorithm

Algorithm 5 Simple DES Encryption

Require: Plaintext P, key K
Ensure: Ciphertext C
1: Generate subkeys K_1 and K_2
2: Apply initial permutation to P
3: Apply first round using K_1
4: Perform swap operation
5: Apply second round using K_2
6: Apply inverse initial permutation
7: **return** ciphertext

3.1.4 Implementation

The SDES prototype accepts input data and a secret key, performs the encryption operation and produces ciphertext. The ciphertext is then processed using the decryption procedure to recover the original plaintext.

3.1.5 Result

The SDES encryption and decryption operations were successfully implemented.

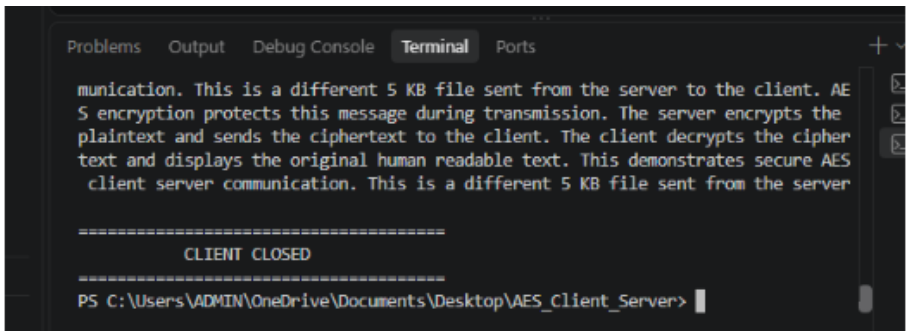

Figure 3.1: SDES Implementation and Output

3.2 Advanced Encryption Standard (AES)

3.2.1 Definition

Advanced Encryption Standard (AES) is a symmetric block cipher used to protect digital information. AES operates on blocks of 128 bits and supports key sizes of 128, 192 and 256 bits.

AES consists of multiple transformation stages including SubBytes, ShiftRows, MixColumns and AddRoundKey.

3.2.2 Mathematical Notation

The AES state can be represented as a 4×4 byte matrix:

$$S = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

The basic encryption relation is:

$$C = AES_K(P)$$

3.2.3 AES Transformations

SubBytes

SubBytes replaces each byte of the state using the AES S-box.

$$S'_{i,j} = SBOX(S_{i,j})$$

ShiftRows

The rows of the state matrix are cyclically shifted by different offsets.

MixColumns

Each column is transformed using matrix multiplication over the finite field $GF(2^8)$.

AddRoundKey

The state is combined with the round key using XOR:

$$S' = S \oplus K_r$$

3.2.4 Algorithm

Algorithm 6 AES Encryption

Require: Plaintext P, key K
Ensure: Ciphertext C

```
Expand key K into round keys
Add initial round key
for each regular round do
    Apply SubBytes
    Apply ShiftRows
    Apply MixColumns
    Apply AddRoundKey
end for
Apply final SubBytes
Apply final ShiftRows
Apply final AddRoundKey
return ciphertext
```

3.2.5 Implementation

AES was implemented to perform encryption and decryption of input data. The implementation generates a secret key and uses AES encryption to transform plaintext into ciphertext. The ciphertext is then decrypted to recover the original data.

3.2.6 Result

The AES implementation successfully encrypted and decrypted the input data.

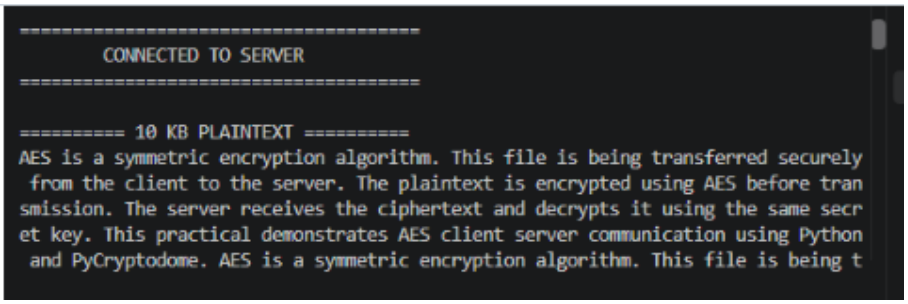

Figure 3.2: AES Implementation and Output

3.3 Stream Cipher (RC4)

3.3.1 Definition

RC4 is a stream cipher that generates a pseudorandom keystream and combines it with plaintext using the XOR operation.

3.3.2 Mathematical Notation

Encryption:

$$C_i = P_i \oplus K_i$$

Decryption:

$$P_i = C_i \oplus K_i$$

where K_i represents the generated keystream byte.

3.3.3 Algorithm

Algorithm 7 RC4 Encryption

Require: Plaintext P, secret key K
Ensure: Ciphertext C
1: Initialize state array S
2: Perform Key Scheduling Algorithm (KSA)
3: Generate keystream using PRGA
4: **for** each plaintext byte P_i **do**
5: $\quad C_i \leftarrow P_i \oplus K_i$
6: **end for**
7: **return** ciphertext

3.3.4 Result

The RC4 implementation successfully generated a keystream and performed XOR-based encryption and decryption.

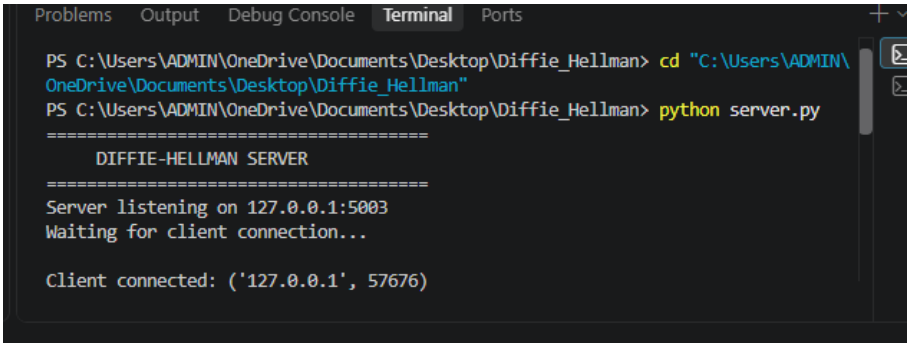

Figure 3.3: RC4 Stream Cipher Implementation and Output

Chapter 4

Public-Key Cryptographic Algorithms

4.1 Diffie–Hellman Key Exchange

4.1.1 Definition

Diffie–Hellman is a key exchange technique that allows two parties to establish a shared secret key over an insecure communication channel without directly transmitting the secret key.

4.1.2 Mathematical Notation

Let p be a prime number and g be a primitive root modulo p.

Alice selects private key a and computes:

$$A = g^a \bmod p$$

Bob selects private key b and computes:

$$B = g^b \bmod p$$

The shared secret calculated by Alice is:

$$K = B^a \bmod p$$

The shared secret calculated by Bob is:

$$K = A^b \bmod p$$

Both values are equal:

$$B^a \bmod p = A^b \bmod p$$

4.1.3 Algorithm

Algorithm 8 Diffie–Hellman Key Exchange

Require: Public values p and g

1: Alice selects private key a
2: Bob selects private key b
3: Alice computes $A \leftarrow g^a \bmod p$
4: Bob computes $B \leftarrow g^b \bmod p$
5: Alice sends A to Bob
6: Bob sends B to Alice
7: Alice computes $K_A \leftarrow B^a \bmod p$
8: Bob computes $K_B \leftarrow A^b \bmod p$
9: Verify $K_A = K_B$

4.1.4 Result

Both parties successfully generated the same shared secret without transmitting the secret itself.

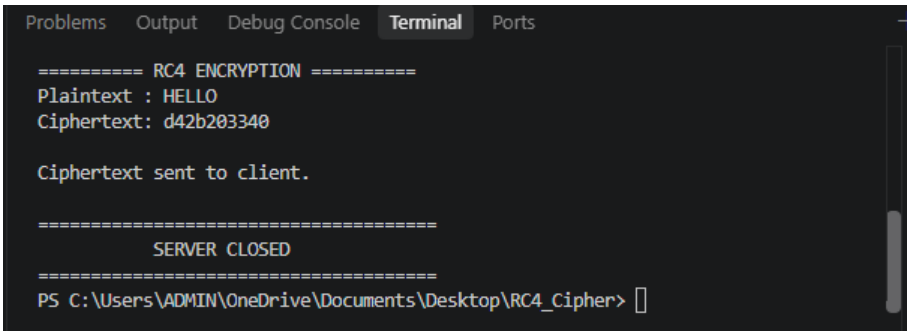

Figure 4.1: Diffie–Hellman Key Exchange Output

4.2 RSA

4.2.1 Definition

RSA is a public-key cryptographic algorithm based on the computational difficulty of factoring large integers. It uses a pair of keys: a public key for encryption and a private key for decryption.

4.2.2 Key Generation

Two prime numbers p and q are selected.

$$n = pq$$

Euler's totient is:

$$\phi(n) = (p-1)(q-1)$$

A public exponent e is selected such that:

$$\gcd(e, \phi(n)) = 1$$

The private exponent d satisfies:

$$ed \equiv 1 \pmod{\phi(n)}$$

The public key is (e, n) and the private key is (d, n).

4.2.3 Encryption

For plaintext M:

$$C = M^e \bmod n$$

4.2.4 Decryption

For ciphertext C:

$$M = C^d \bmod n$$

4.2.5 Algorithm

Algorithm 9 RSA

Require: Two prime numbers p, q

1: Compute $n \leftarrow pq$
2: Compute $\phi(n) \leftarrow (p-1)(q-1)$
3: Select e such that $\gcd(e, \phi(n)) = 1$
4: Compute d such that $ed \equiv 1 \pmod{\phi(n)}$
5: Public key $\leftarrow (e, n)$
6: Private key $\leftarrow (d, n)$
7: Encrypt using $C \leftarrow M^e \bmod n$
8: Decrypt using $M \leftarrow C^d \bmod n$
9: **return** plaintext

4.2.6 Result

The RSA prototype successfully generated public and private keys and performed encryption and decryption.

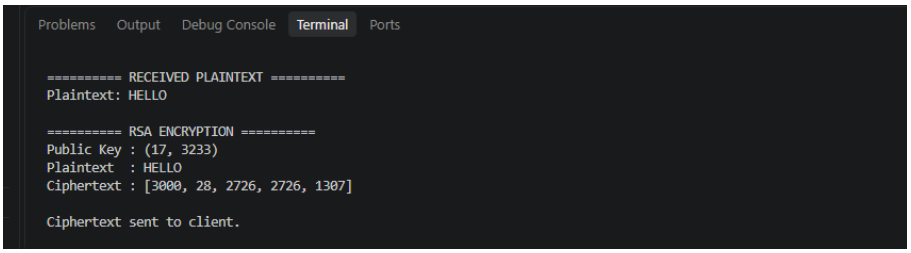

Figure 4.2: RSA Implementation and Output

Chapter 5

Implementation and Results

The prototype includes implementations of nine cryptographic algorithms. Each algorithm was tested using suitable plaintext, keys and parameters. The encryption output was verified by performing the corresponding decryption operation.

Table 5.1: Summary of Implemented Algorithms

No.	Algorithm	Type	Result
1	Caesar Cipher	Classical	Successful
2	Playfair Cipher	Classical	Successful
3	Hill Cipher	Classical	Successful
4	Vigenère Cipher	Classical	Successful
5	SDES	Symmetric	Successful
6	AES	Symmetric	Successful
7	RC4	Stream Cipher	Successful
8	Diffie–Hellman	Key Exchange	Successful
9	RSA	Asymmetric	Successful

5.1 Verification

The implementations were verified by comparing the decrypted output with the original plaintext. The expected condition is:

$$D_K(E_K(P)) = P$$

For key exchange algorithms, the generated shared keys were compared:

$$K_A = K_B$$

The results demonstrate that the implemented algorithms performed the expected cryptographic operations.

Chapter 6

Screenshots

This chapter contains screenshots of the prototype implementations and their execution outputs.

6.1 Caesar Cipher

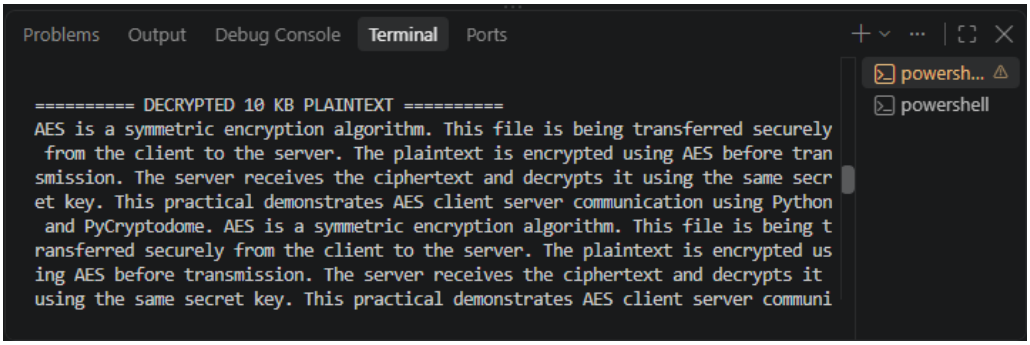

Figure 6.1: Caesar Cipher Execution

6.2 Playfair Cipher

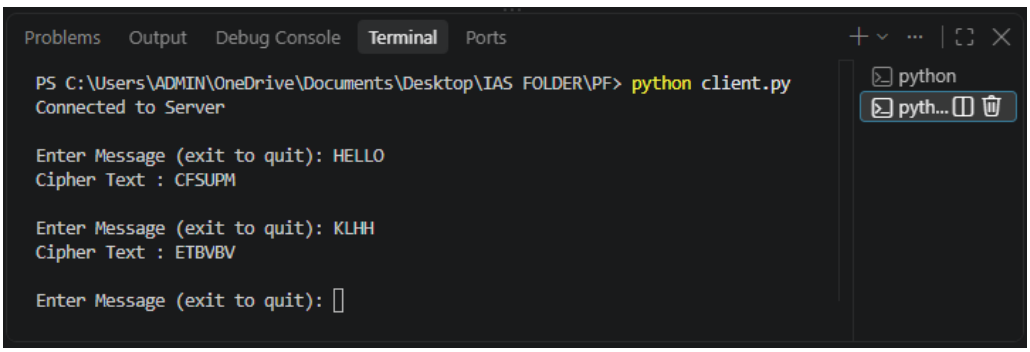

Figure 6.2: Playfair Cipher Execution

6.3 Hill Cipher

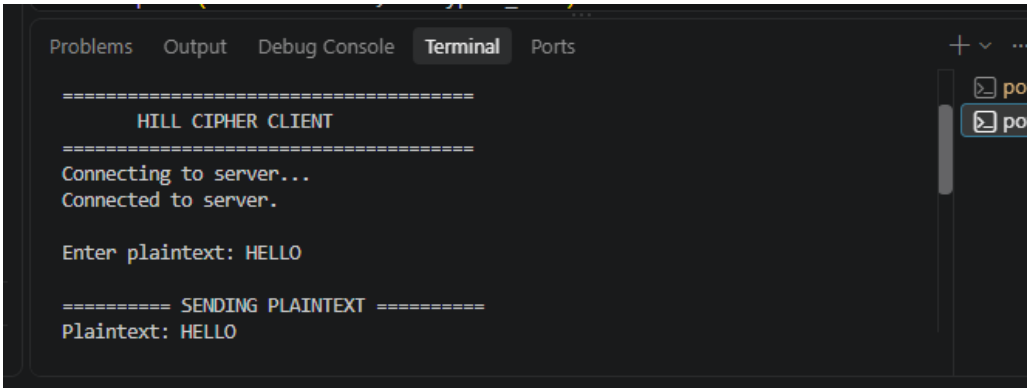

Figure 6.3: Hill Cipher Execution

6.4 Vigenère Cipher

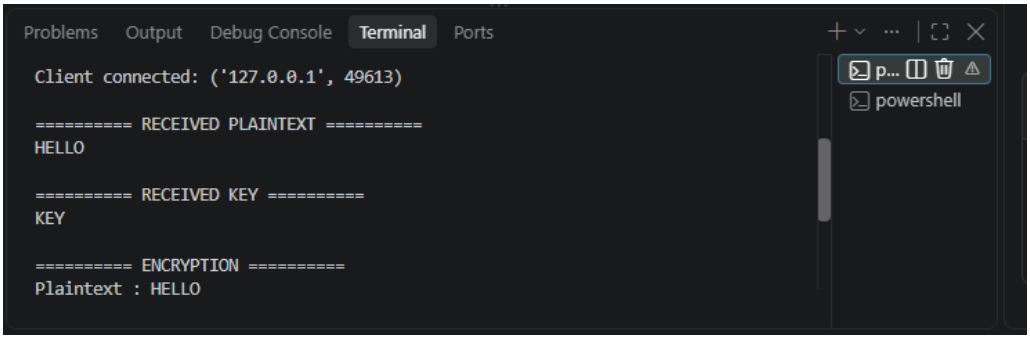

Figure 6.4: Vigenère Cipher Execution

6.5 SDES

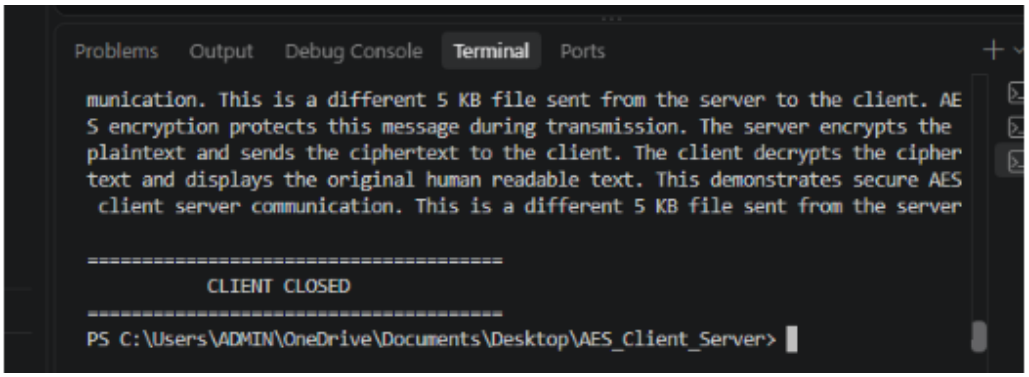

Figure 6.5: SDES Execution

6.6 AES

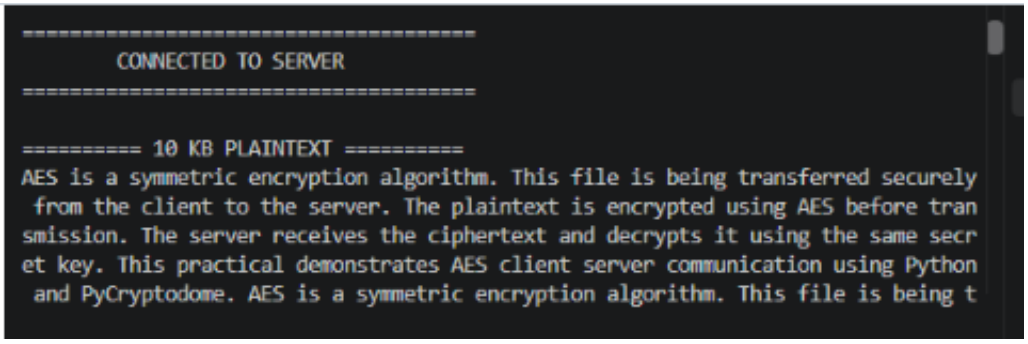

Figure 6.6: AES Execution

6.7 RC4

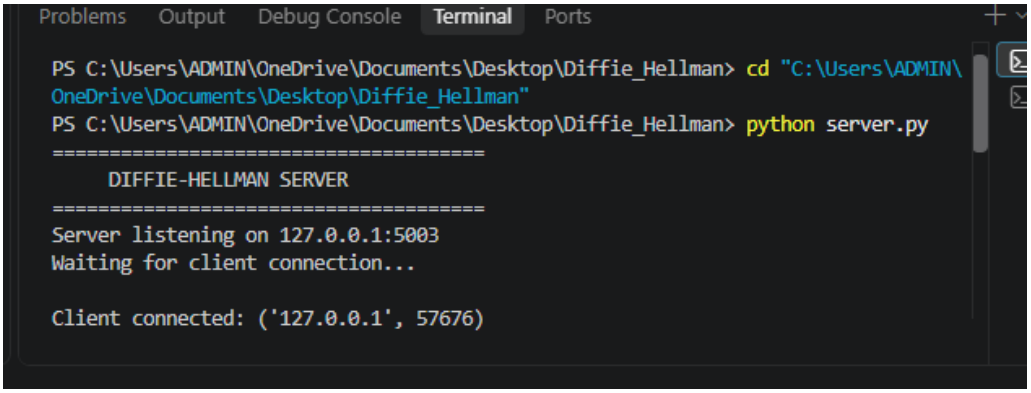

Figure 6.7: RC4 Execution

6.8 Diffie–Hellman

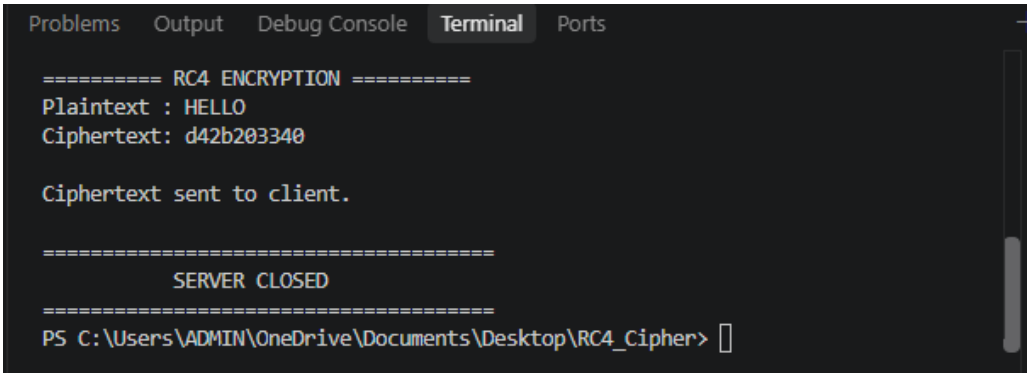

Figure 6.8: Diffie–Hellman Execution

6.9 RSA

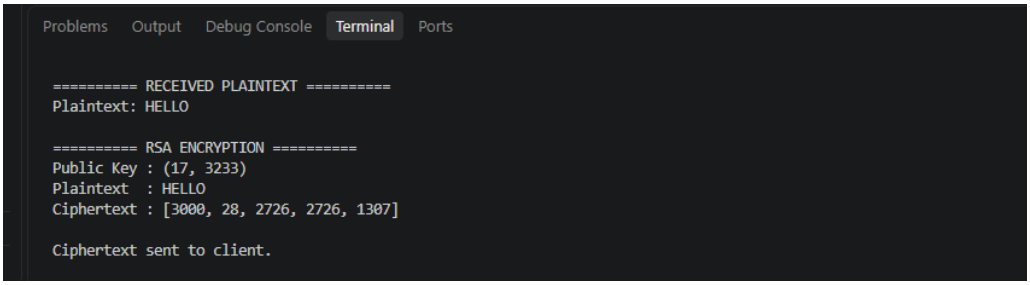

Figure 6.9: RSA Execution

Chapter 7

Conclusion

The prototype provided practical exposure to the concepts of information assurance and cryptography. Nine cryptographic algorithms were studied and implemented, including classical ciphers, symmetric-key algorithms, stream ciphers and public-key cryptographic techniques.

The implementation demonstrated the transformation of plaintext into ciphertext and the recovery of plaintext through decryption. The mathematical formulations and algorithms provided an understanding of how cryptographic techniques operate internally.

The practical implementation also helped in understanding the differences between symmetric and asymmetric cryptography, key generation, encryption, decryption and secure key exchange.

Chapter 8

GitHub Repository

The source code, implementation files and related project resources are available in the GitHub repository below.

GitHub URL:

https://github.com/Luhitha-1681/IAS---INSEM1--PROJECT-

Bibliography

[1] William Stallings,
Cryptography and Network Security: Principles and Practice,
Pearson.

[2] Jonathan Katz and Yehuda Lindell,
Introduction to Modern Cryptography,
CRC Press.

[3] National Institute of Standards and Technology,
Advanced Encryption Standard (AES),
FIPS PUB 197.

[4] R. L. Rivest, A. Shamir and L. Adleman,
A Method for Obtaining Digital Signatures and Public-Key Cryptosystems,
Communications of the ACM.